Transfer Learning & the DocScan Capstone

Week 16 — Session 1: Standing on a giant's shoulders

Practical Image Processing

Lecture 16.1 ▪ 60–70 min

Where we stand

- Week 12: HOG+SVM plateaued at **0.48** on look-alike cars.
- Week 13: CNNs learn features end-to-end — but need lots of data.
- Today: **transfer learning** — reuse a net trained on millions of images so a tiny dataset still gets deep-learning power.

Today's real result

Same cars, same SVM, but **deep features** from a pretrained net: **0.48** $\rightarrow$ **0.87**.

Roadmap

The small-data problem

The key insight: reuse features

Two flavours of transfer learning

The feature-extraction pipeline

Our result

What transfers, and how far

Squeezing more from small data

In-class exercise

Next session

The small-data problem

Why we can't just train a big CNN

- Deep CNNs have millions of parameters and want 10^4–10^6 labelled images.
- We have **60 images per car model** — training a big net from scratch would memorize, not generalize.
- Yet the HOG ceiling (0.48) proves hand features aren't enough. What now?

Key idea

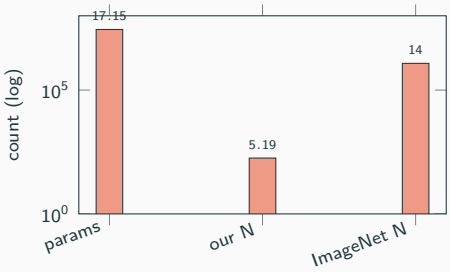
Most real projects have *small* data. Transfer learning is how you still use deep learning.

Parameters vs. examples: a feel for the gap

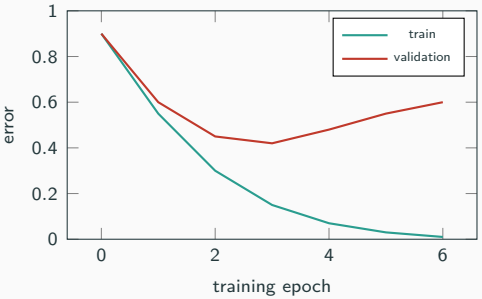
Rough rule

You want at least a handful of examples *per free parameter*. A net with P parameters and N examples is in danger when $N \ll P$.

- ConvNeXt-Tiny: $P \approx 28{,}000{,}000$.
- Our data: $N = 180$ images total.
- $N/P \approx 6 \times 10^{-6}$ — hopeless from scratch.



What overfitting looks like



- Training error keeps dropping toward zero.
- Validation error bottoms out, then *rises* — the net memorizes noise.
- With 180 images this happens in a few epochs.

✕ Pitfall

A tiny gap on train tells you nothing. Always watch the *validation* curve.

The key insight: reuse features

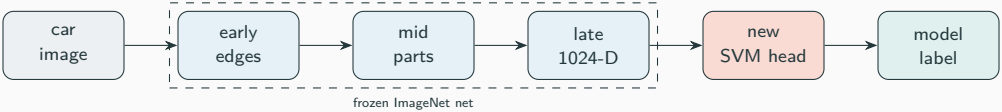
The key insight

- A CNN trained on ImageNet (1.2M images, 1000 classes) learned *general* visual features: edges, textures, shapes, parts.
- Those early/mid features are useful for *almost any* image task — cars included.
- So: **reuse** those learned features instead of learning them from scratch.

Key idea

The first layers of a good vision net are a general-purpose feature extractor. Don't waste them.

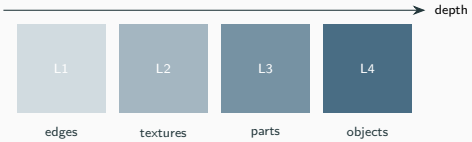
Anatomy of reuse



- The frozen net is a *fixed function* ϕ : image $\to \mathbb{R}^{1024}$.
- We only train the small orange head on top — seconds, not hours.

Features are learned, not designed

- Week 12 HOG: *we* decided to count edge orientations in cells.
- A trained net *discovered* its own detectors from data — and better ones.
- Visualizing them shows a clear hierarchy from edges to object parts.



Same as W6

Early filters look like the Gabor/edge kernels we built by hand in the filtering weeks.

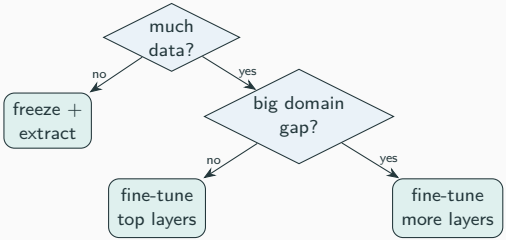
Two flavours of transfer learning

Feature extraction vs. fine-tuning

Approach	What you do	When
Feature extraction	freeze the net, train a new classifier head	very small data
Fine-tuning	unfreeze some top layers, train them too	more data

- Today we use **feature extraction**: a frozen ImageNet ConvNeXt turns each car into a 1024-D vector, then an SVM classifies.
- Exactly Week 12's classifier — better features, nothing else changed.

A decision rule you can carry



- Small data → never touch the backbone; you would just overfit it.
- More data + similar domain → fine-tune a few top layers.
- More data + distant domain → fine-tune deeper.

✔ Good practice

When unsure, start frozen. It is faster, safer, and a strong baseline.

The feature-extraction pipeline

Extracting deep features (code)

```python
import torch, timm
from torchvision import transforms as T

# 1. load a pretrained backbone, drop its classifier
net = timm.create_model("convnext_tiny", pretrained=True,
                        num_classes=0)      # num_classes=0 -> feature vector
net.eval()                                   # freeze: no training

# 2. use the model's OWN preprocessing (must match!)
prep = T.Compose([T.Resize(236), T.CenterCrop(224),
                  T.ToTensor(),
                  T.Normalize(mean=[0.485,0.456,0.406],
                              std =[0.229,0.224,0.225])])

@torch.no_grad()
def features(img):                           # img: PIL RGB
    x = prep(img).unsqueeze(0)               # 1x3x224x224
    return net(x).squeeze(0).numpy()         # -> 1024-D vector
```

From vectors to a classifier (code)

```python
import numpy as np
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = np.stack([features(im) for im in train_images])  # (180,1024)
y = np.array(train_labels)

clf = make_pipeline(StandardScaler(),
                    SVC(C=10, kernel="rbf"))          # same SVM as Week 12
clf.fit(X, y)

Xt = np.stack([features(im) for im in test_images])
print("accuracy:", clf.score(Xt, test_labels))        # ~0.87
```

- The *only* change from Week 12 is where `X` comes from.

Why standardize the features?

Per-dimension scaling

For each feature dimension j:

$$\tilde{x}_j = \frac{x_j - \mu_j}{\sigma_j}$$

with μ_j, σ_j estimated on the *training* set only.

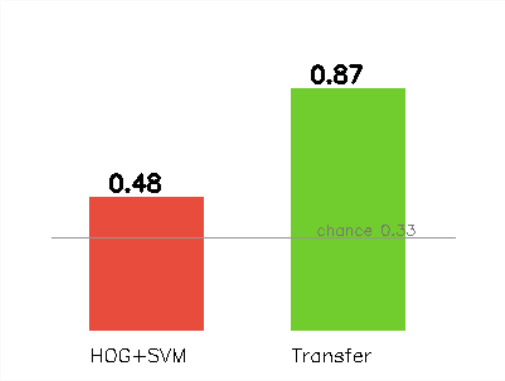
- Deep features have wildly different scales per dimension.
- An RBF-SVM measures distance; unscaled dims dominate it.
- Fit the scaler on train, apply to test — never peek at test stats.

✕ Pitfall

Fitting the scaler on all data (train+test) leaks information and inflates your score.

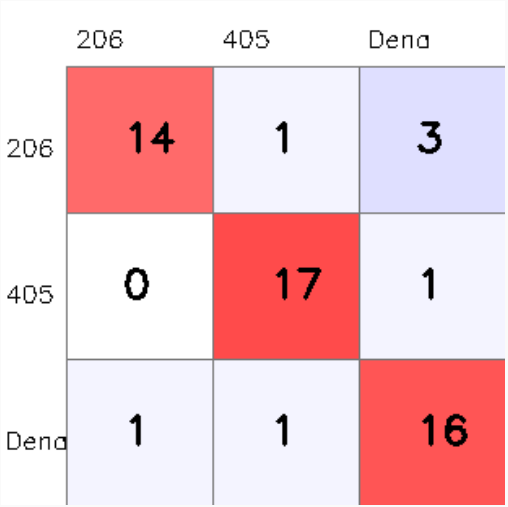
Our result

The ceiling lifts



- **Same** 180 images, **same** SVM, **same** split.
- Only change: HOG $\rightarrow$ frozen ImageNet features.
- 0.48 $\rightarrow$ **0.87**. The features were the bottleneck all along.

Confusion matrix after transfer



- The diagonal is now dominant — most cars classified correctly.
- Compare to Week 12's near-random confusion matrix.
- The pretrained net "sees" the fine differences HOG missed.

A worked accuracy number

Reading the matrix

With 3 classes $\times$ 20 test images $=$ 60 tests, suppose the diagonal sums to 52:

$$\text{acc} = \frac{52}{60} = 0.867 \approx 0.87$$

- Per-class recall = (row diagonal) / (row total).
- One weak class can hide inside a good average — always read per-class.

	predicted		
true	A	B	C
A	**18**	1	1
B	2	**17**	1
C	1	2	**17**

recall: A .90, B .85, C .85

Why this works so well

- ImageNet includes many vehicles — the net already learned car-relevant parts.
- Its mid-level features (grille shapes, light patterns) are exactly what separates models.
- We get the power of a huge net with only 60 images per class and seconds of training.

✓ Good practice

Transfer learning is the single most useful deep-learning technique for real, small-data projects.

What transfers, and how far

What each layer transfers

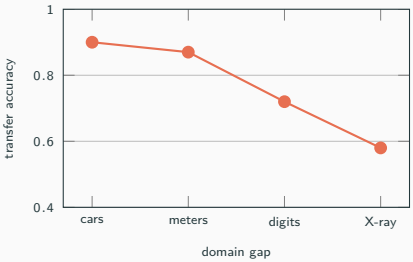
- **Early layers**: edges, colours, textures — universal, transfer to *any* task.
- **Middle layers**: parts and motifs — transfer well to related domains.
- **Late layers**: class-specific ("golden retriever") — least transferable; often replaced.

Key idea

The more your task differs from ImageNet, the more late layers you should retrain.

The domain gap

- ImageNet is natural photos. Very different images (X-rays, thermal, tiny digits) transfer more weakly.
- Meters and cars are ordinary photos → small gap → transfer works great (our 0.87).
- Larger gap → fine-tune more layers, or find a model pretrained on closer data.



Math

Rule of thumb: small gap + small data → freeze & extract; large gap + more data → fine-tune.

Squeezing more from small data

Data augmentation for small data

- Multiply your data for free: random flips, small rotations, brightness/contrast jitter, crops.
- Each variant teaches the same label under a new nuisance $\rightarrow$ more invariance, less overfitting.
- Especially valuable when combined with transfer learning on tiny datasets.

✔ Good practice

Augmentation is the cheapest accuracy you will ever buy — always try it on small-data problems.

Augmentation, in practice (code)

```python
aug = T.Compose([
    T.RandomHorizontalFlip(p=0.5),
    T.RandomRotation(8),                  # small: a car is still a car
    T.ColorJitter(brightness=0.2, contrast=0.2),
    T.RandomResizedCrop(224, scale=(0.8, 1.0)),
])
# make 5 augmented copies per training image, extract features on each
X = np.stack([features(aug(im)) for im in train_images for _ in range(5)])
```

✕ Pitfall

Only augment *training* data, and only with label-preserving transforms — a vertical flip of a digit “6” is a lie.

Practical transfer-learning tips

- Use the pretrained model's *own* preprocessing (resize, normalize) — mismatch hurts a lot.
- Start with **feature extraction**; only fine-tune if you have enough data and need more.
- Freeze early layers (general), fine-tune late layers (task-specific) if you do fine-tune.
- Watch for overfitting — small data still bites; use augmentation and regularization.

In-class exercise

In-class exercise (10 min)

Reason it through

1. You have 40 images per class. From-scratch CNN or transfer learning? Why?
2. Transfer learning lifted cars 0.48→0.87 with the *same* SVM. What changed, exactly?
3. You standardize features on train+test together and your score jumps. Trust it? Why not?
4. Your input is 128×64 but the backbone expects 224×224. What must you do first?

Debrief

- (1) **Transfer learning** — 40 images is far too few to train a deep net from scratch without overfitting.
- (2) Only the **features**: hand-designed HOG $\rightarrow$ pretrained deep features. The classifier was identical.
- (3) **No** — fitting the scaler on test data leaks information; the gain is fake.
- (4) **Resize/normalize** with the backbone's own preprocessing before extracting.

Key idea

Good ML engineering is mostly good *features* and honest evaluation — not fancier models.

Next session

Next session: shipping DocScan

- We can now *read* the digits with transfer learning. Next: assemble the **whole DocScan pipeline** end to end.
- Every week of the course was one stage — we bolt them together.
- Then: engineering for the real world (confidence gates, fallbacks) and the **capstone deliverable**.

Come ready

Re-skim your Week 11 homography and Week 9 contour code — you'll wire them in.

Better features, not a bigger model.

A frozen giant did the seeing; a tiny SVM did the deciding — 0.48 to 0.87.